



CHATME API

Socket.IO Testing with Postman

Student Testing Guide

Authentication, presence, typing, messages, and durable receipts

PRESENCE

TYPING

MESSAGES

RECEIPTS

Contents

Use this guide as a practical lab: prepare two users, connect two Socket.IO tabs, and verify each realtime behavior end to end.

1. Import the environment	3
2. Create and save the Socket.IO request	3
3. Add event listeners	4
message.created	4
receipt.delivered and receipt.read	5
presence.changed	5
typing.started	5
typing.stopped	5
4. Emit client commands	5
presence.subscribe	6
presence.unsubscribe	6
typing.start	6
typing.stop	7
5. Complete two-user test	7
Limits and troubleshooting	8

Important Postman limitation

Socket.IO requests belong in a separate multi-protocol collection. Use the REST API or Swagger for login, message writes, and receipt writes.

This guide creates a reusable Postman Socket.IO request for the ChatMe `/chat` namespace. Use the Postman desktop app because its Socket.IO client supports named events, JSON arguments, listeners, and acknowledgement callbacks.

Postman stores Socket.IO requests in a multi-protocol collection. That collection must be separate from the HTTP collection used for login, message history, message sending, and receipt updates.

1. Import the environment

Import [ChatMe-Socket.IO.postman_environment.json](#) into Postman and select the ChatMe Socket.IO environment.

Set these environment values:

Variable	Value
<code>apiBaseUrl</code>	Local: <code>http://localhost:3000/v1</code> ; production: <code>https://HOST/v1</code>
<code>socketUrl</code>	Local: <code>ws://localhost:3000/chat</code> ; production: <code>wss://HOST/chat</code>
<code>accessTokenA</code>	Current access token for the first test user
<code>accessTokenB</code>	Current access token for the second test user
<code>conversationId</code>	A direct conversation shared by users A and B
<code>throughMessageId</code>	An incoming message ID used when testing delivered and read receipts

Replace `HOST` with the exact deployment hostname. `WSS` means WebSocket Secure: it is TLS-encrypted and is the socket equivalent of HTTPS. The companion environment file remains available in the repository at [apps/api/postman/ChatMe-Socket.IO.postman_environment.json](#).

Keep access tokens as local/secret values. Do not commit real tokens or share them in a Postman workspace.

When API documentation is enabled, use Swagger at `{{apiBaseUrl}}/docs` to obtain the prerequisites. Otherwise, call the same REST endpoints from an HTTP Postman collection:

1. Sign in two different users and copy each `accessToken`.
2. Create or open one direct conversation between them.
3. Copy its `id` into `conversationId`.

2. Create and save the Socket.IO request

1. In Postman, select New > Socket.IO. Do not select raw WebSocket.
2. Enter `{{socketUrl}}`.
3. Before connecting, add the header `Authorization: Bearer {{accessTokenA}}`.
4. In Settings, use Socket.IO client version v4 and keep the handshake path as `/socket.io`. For a sleeping Render free service, set the handshake timeout to about `90000` milliseconds.
5. Select Connect.
6. Save the request as User A - Chat socket in a new multi-protocol collection named ChatMe Socket.IO.

For two-user tests, duplicate the request in another tab, change its header to `Authorization: Bearer {{accessTokenB}}`, connect it, and save it as User B - Chat socket.

The namespace belongs in `socketUrl` as `/chat`. The handshake path remains `/socket.io`; it must not be changed to `/chat`.

An invalid, expired, or revoked token fails the connection. Its `connect_error.data` has this shape (Postman may display it inside a larger connection error):

```
{
  "code": "AUTH_ACCESS_TOKEN_INVALID",
  "message": "A valid access token is required."
}
```

3. Add event listeners

All UUIDs and timestamps below are examples. Actual values and participant ordering depend on the accounts, conversation, and server time used for the test.

Open Events, add each server event below, and select Listen for all six:

Event	When it is received
<code>message.created</code>	A participant persists a new message through the REST API
<code>receipt.delivered</code>	A participant advances the durable delivered frontier through REST
<code>receipt.read</code>	A participant advances the durable read frontier through REST
<code>presence.changed</code>	A subscribed participant changes between online and offline
<code>typing.started</code>	Another subscribed participant starts or refreshes typing
<code>typing.stopped</code>	Another subscribed participant stops typing or its timer expires

`message.created`

```
{
  "id": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "clientMessageId": "74bd9a1d-3b48-49da-87cc-6a79e77140a3",
  "senderId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "kind": "text",
  "text": "Hello from Postman",
  "createdAt": "2026-08-12T16:30:00.000Z"
}
```

receipt.delivered and receipt.read

Both events carry the participant's complete receipt state - the latest incoming message recorded as delivered and read. The `read` field can be `null`. Clients apply the newest `version` without losing an earlier update.

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "throughMessageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
  "at": "2026-08-12T16:31:00.000Z",
  "version": 2,
  "delivered": {
    "messageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
    "at": "2026-08-12T16:30:30.000Z"
  },
  "read": {
    "messageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
    "at": "2026-08-12T16:31:00.000Z"
  }
}
```

presence.changed

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "status": "online",
  "occurredAt": "2026-08-12T16:32:00.000Z"
}
```

typing.started

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "expiresAt": "2026-08-12T16:32:05.000Z"
}
```

typing.stopped

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "occurredAt": "2026-08-12T16:32:04.000Z"
}
```

4. Emit client commands

For each command:

1. Enter the exact event name next to Send.
2. Add one JSON argument containing the payload below.
3. Turn on Ack.
4. Select Send and inspect the acknowledgement in the response pane.

presence.subscribe

```
{
  "conversationId": "{{conversationId}}"
}
```

Successful acknowledgement:

```
{
  "ok": true,
  "data": {
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
    "participants": [
      {
        "userId": "0ab53389-d1c7-4903-9c59-ceb5b8a5dbaf",
        "status": "online"
      },
      {
        "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
        "status": "online"
      }
    ]
  },
  "typing": []
}
```

presence.unsubscribe

```
{
  "conversationId": "{{conversationId}}"
}
```

Successful acknowledgement:

```
{
  "ok": true,
  "data": {
    "conversationId": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

typing.start

```
{
  "conversationId": "{{conversationId}}"
}
```

Successful acknowledgement:

```
{
  "ok": true,
  "data": {
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
    "expiresAt": "2026-08-12T16:32:05.000Z"
  }
}
```

Typing expires after five seconds. A real client refreshes `typing.start` about every three seconds while input continues.

`typing.stop`

```
{
  "conversationId": "{{conversationId}}"
}
```

Successful acknowledgement:

```
{
  "ok": true,
  "data": {
    "conversationId": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

Failed commands use the same acknowledgement envelope:

```
{
  "ok": false,
  "error": {
    "code": "REALTIME_CONVERSATION_NOT_FOUND",
    "message": "Conversation was not found."
  }
}
```

Possible codes are `AUTH_ACCESS_TOKEN_INVALID`, `REALTIME_PAYLOAD_INVALID`, `REALTIME_CONVERSATION_NOT_FOUND`, `REALTIME_RATE_LIMITED`, and `REALTIME_INTERNAL_ERROR`.

5. Complete two-user test

1. Connect user A and user B in separate Socket.IO tabs.
2. Send `presence.subscribe` from both tabs with the same `conversationId`.
3. Send `typing.start` from A. B receives `typing.started`; A does not receive its own typing event.
4. Send `typing.stop` from A. B receives `typing.stopped`.
5. In an HTTP request or Swagger, send a message as A with `POST {{apiBaseUrl}}/conversations/{{conversationId}}/messages`, the header `Authorization: Bearer {{accessTokenA}}`, and this body:

```
{
  "clientMessageId": "7d444840-9dc0-41d1-b245-5ffdc74fad2",
  "text": "Hello from Postman"
}
```

Use a fresh `clientMessageId` UUID for each new test. Both socket tabs receive one `message.created` event. Copy the returned server message `id` into `throughMessageId`.

6. As B, call `PUT {{apiBaseUrl}}/conversations/{{conversationId}}/receipts/delivered` with `Authorization: Bearer {{accessTokenB}}` and `{ "throughMessageId": "{{throughMessageId}}" }`. The selected message must be incoming for B, not one B sent. Both tabs receive `receipt.delivered`.
7. Repeat with `/receipts/read` using B's token. Both tabs receive `receipt.read`.
8. Disconnect every active B socket, including other Postman tabs or mobile sessions. After the ten-second reconnection grace period, A receives `presence.changed` with `status: "offline"`.

Message sending and receipt mutation deliberately remain REST operations. There is no client-to-server `message.send`, `receipt.delivered`, or `receipt.read` Socket.IO command. Socket events notify connected clients only after durable database writes succeed.

Add listeners before triggering REST requests because socket events are not replayed. Presence and typing events require an active `presence.subscribe`; message and receipt events do not. Receipt boundaries must reference an incoming message, and replaying an unchanged receipt produces no new event.

Limits and troubleshooting

- A socket may subscribe to at most 20 conversations.
- A user may have at most five active socket connections by default.
- The server accepts 30 realtime commands per 10-second window per socket.
- Access tokens are short-lived. The server closes a socket when its token expires; refresh or sign in again, then reconnect with the new access token.
- Command payloads are strict and may contain only `conversationId`.
- Missing and unauthorized conversations return the same error intentionally.
- Production connections use `wss://`, not `https://` or `ws://`.
- Postman Socket.IO supports WebSocket transport, not long polling. ChatMe supports the WebSocket transport used here.
- If Postman shows a connection error, open the Postman Console and verify the URL, `/chat` namespace, `/socket.io` handshake path, and Authorization header.
- On the Render free tier, the first connection can take longer while the service wakes from inactivity.

Official Postman references:

- [Create a Socket.IO request](#)
- [Listen to Socket.IO events](#)
- [Work with WebSocket messages and Socket.IO arguments](#)